

Projektowanie i analiza algorytmów	
Kierunek <i>Informatyczne Systemy Automatyki</i>	Termin <i>Czwartek TP 13¹⁵ – 14⁵⁵</i>
Imię, nazwisko, numer albumu <i>Marcel Galiński 284158</i>	Data <i>22.04</i>



Politechnika
Wrocławska

SPRAWOZDANIE – MINIPROJEKT NR 1

Cel ćwiczeń:

Implementacja oraz analiza porównawcza trzech algorytmów sortowania: przez scalanie (Merge Sort), szybkiego (Quicksort) oraz introspektywnego (Introsort).

Wstęp:

Przykładowe zastosowania wybranych algorytmów:

Systemy operacyjne: Zarządzanie priorytetami procesów i kolejkovanie zadań.

Logistyka i E-commerce: sortowanie wyników wyszukiwania produktów według ceny lub popularności.

Multimedia: Kodeki wideo i audio często wykorzystują algorytmy sortujące w procesach kompresji danych.

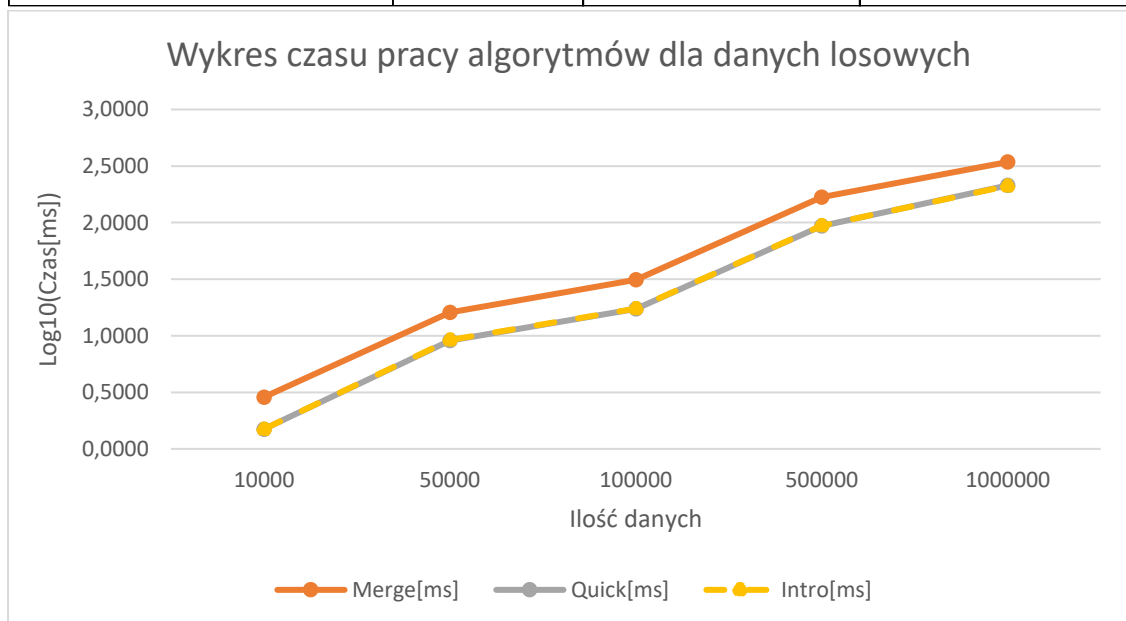
Analiza wybranych algorytmów:

- A. Sortowanie przez scalanie (Merge Sort):
Opiera się na paradygmacie „dziel i rządz”. Tablica jest dzielona rekurencyjnie na połowy, aż do uzyskania fragmentów jednoelementowych. Następnie są one scalane w taki sposób, aby wynikowa tablica była posortowana. Spodziewamy się bardzo stabilnej wydajności niezależnie od stopnia posortowania danych wejściowych. Jest to algorytm "odporny" na złośliwe zestawy danych. Teoretyczna złożoność algorytmu to $O(n \log n)$.
- B. Sortowanie szybkie (Quicksort): Wybiera element osiowy (pivot), a następnie dzieli tablicę na dwie podtablice: elementy mniejsze od pivota oraz większe lub równe. Proces powtarza się rekurencyjnie dla każdej z części. Spodziewamy się w większości przypadków, że będzie najszybszym algorytmem, ale przy tablicach posortowanych w 99,7% lub 100% może drastycznie zwolnić. Teoretyczna złożoność algorytmu to $O(n \log n)$, lecz może dochodzić do n^2 .
- C. Sortowanie introspektywne (Introsort): Łączy Quicksort i Heap Sort. Zaczyna pracę jak Quicksort, ale jeśli jego złożoność przekroczy limit, przełącza się na stabilne i pewne sortowanie przez kopcowanie. Spodziewamy się najlepszej możliwej wydajności przy jednoczesnym bezpieczeństwie. Powinien być tak szybki jak Quicksort na danych losowych, ale znacznie szybszy na danych niekorzystnych. Teoretyczna złożoność algorytmu to: $O(n \log n)$

Wyniki badań:

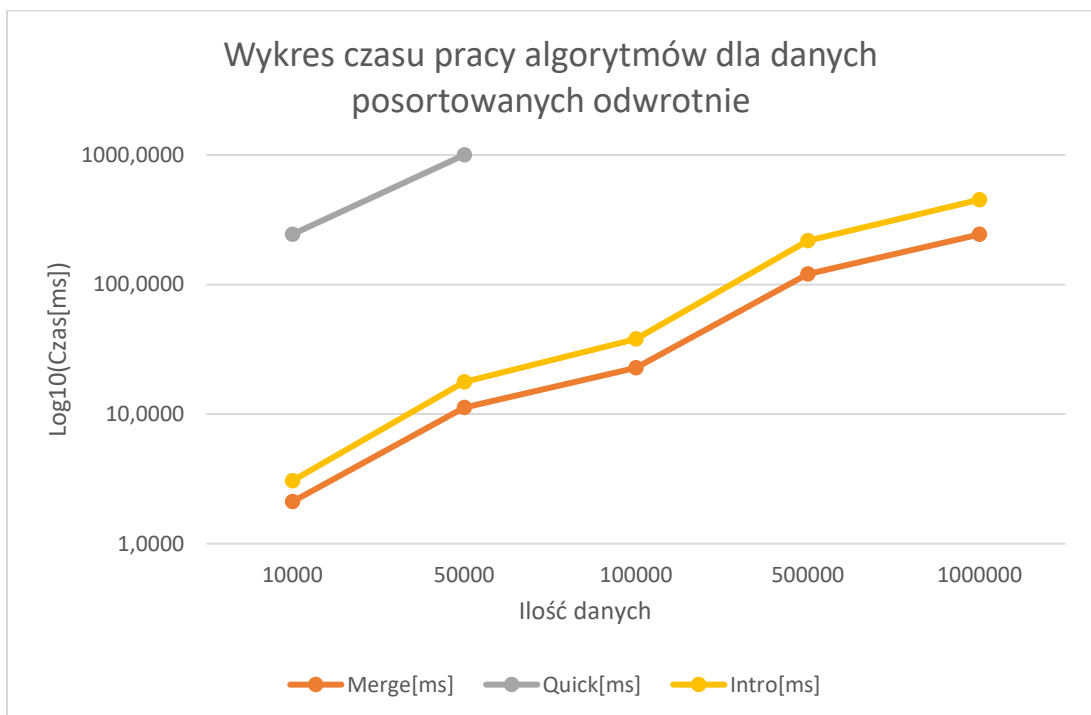
Czas = Czas 100 testów / 100

Dane losowe			
Rozmiar	Merge[ms]	Quick[ms]	Intro[ms]
10000	2,869	1,495	1,495
50000	16,080	9,089	9,209
100000	31,235	17,251	17,376
500000	167,506	93,098	93,822
1000000	343,227	213,315	211,001



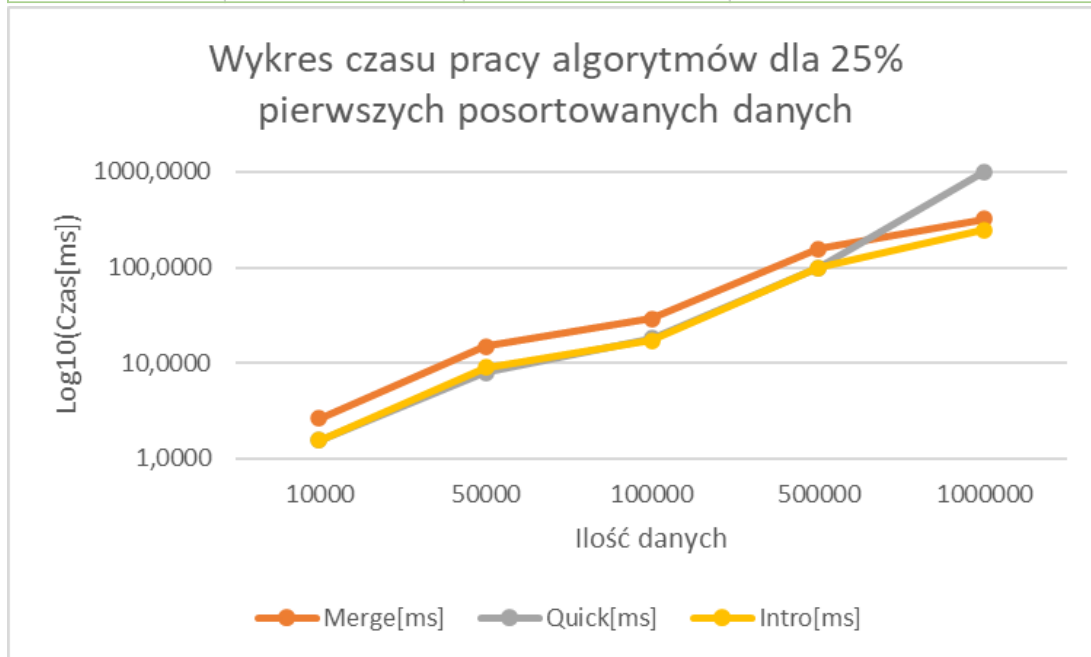
Jak widać dla danych losowych najszybszymi algorytmami są QuickSort, oraz IntroSort. Jest to związane z tym, że pod spodem to ten algorytm. IntroSort dla wszystkich ilości oprócz 1M jest wolniejszy, jest to spowodowane dodatkowym liczeniem głębokości, aby zmienić algorytm na HeapSort. Złożoność Wszystkich algorytmów to $O(n)$.

Dane posortowane odwrotnie			
Rozmiar	Merge[ms]	Quick[ms]	Intro[ms]
10000	2,1055	244,0360	3,0531
50000	11,2100	Limit!	17,7472
100000	22,7056	Limit!	38,0400
500000	120,5720	Limit!	217,1940
1000000	243,6170	Limit!	451,7170



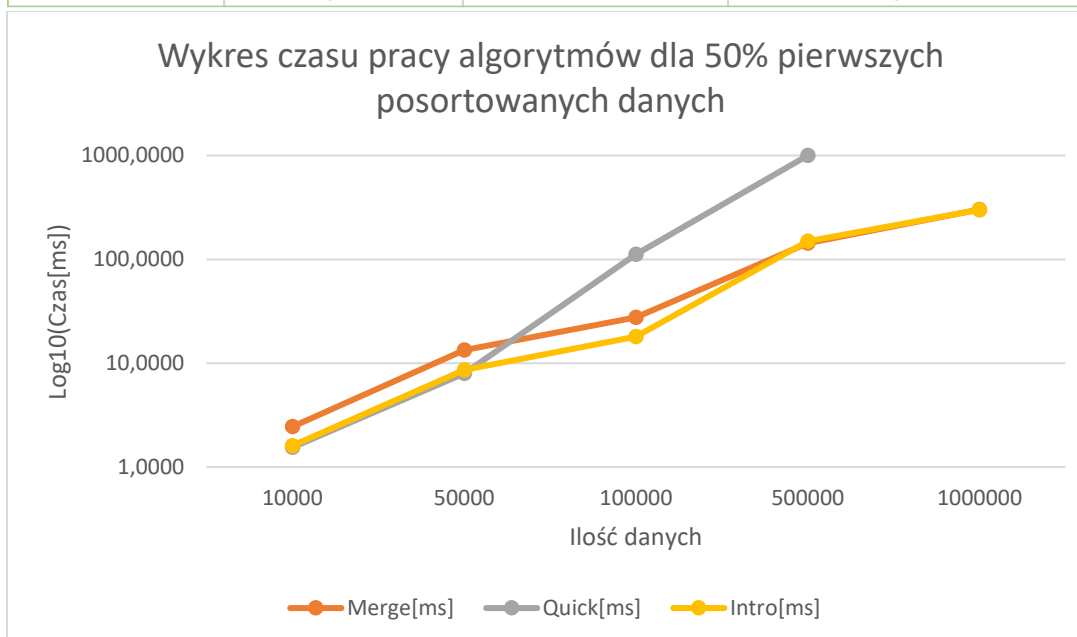
Dla danych posortowanych odwrotnie najszybszy okazał się HeapSort. Warto zauważyć, że limit czasu QuickSort osiągnął już przy 5tyś elementów. Oczywiście jest to związane z bardzo niefortunnie dobranym pivotem(ostatnim elementem brany pod uwagę jako środek). Złożoność MergeSort i IntroSort to $O(n)$, a QuickSort może dochodzić do n^2 .

25% pierwszych danych posortowanych			
Rozmiar	Merge[ms]	Quick[ms]	Intro[ms]
10000	2,6284	1,5422	1,5511
50000	14,9154	7,9643	8,9629
100000	29,2834	18,1323	17,1851
500000	157,2000	99,3380	99,9138
1000000	323,6690	1000,0000	246,4450



Dla 25% pierwszych posortowanych, a danych pozostałych losowych, widzimy, że najszybszy jest QuickSort. Co Ciekawe, już przy milionie elementów Quicksort już wychodzi poza zakres 1s i włącza się Heapsort. Złożoność Wszystkich algorytmów to $O(n)$.

50% pierwszych danych posortowanych			
Rozmiar	Merge[ms]	Quick[ms]	Intro[ms]
10000	2,4484	1,5462	1,6063
50000	13,3710	7,9643	8,6002
100000	27,7018	111,4000	18,0883
500000	143,1750	Limit!	150,0190
1000000	302,1470	Limit!	300,4290



Dla 50% pierwszych posortowanych, a danych pozostałych losowych, widzimy, że najszybszy jest QuickSort do 50tyś elementów, wtedy jeszcze działa, dla 100tyś najszybszy jest HeapSort, a od 500tyś zrównuje się on z MergeSortem. Złożoność MergeSort i IntroSort to $O(n)$, a QuickSort może dochodzić do n^2 .

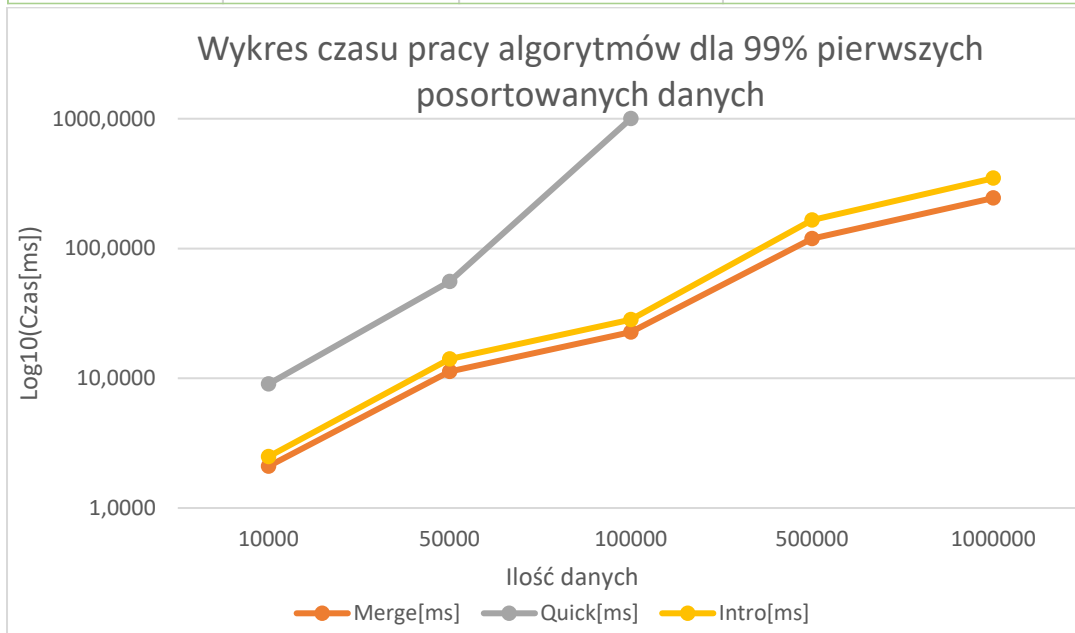
75% pierwszych danych posortowanych			
Rozmiar	Merge[ms]	Quick[ms]	Intro[ms]
10000	2,2742	2,3247	2,1320
50000	12,5288	8,9288	10,1689
100000	25,2718	Limit!	26,1895
500000	130,9610	Limit!	160,3420
1000000	266,5750	Limit!	391,3580

Dla 75% pierwszych posortowanych, a danych pozostałych losowych, widzimy, że najszybszy jest QuickSort do 50tyś elementów, dla 100tyś najszybszy jest HeapSort, a od 500tyś zrównuje się on z MergeSortem. Złożoność MergeSort i IntroSort to $O(n)$, a QuickSort może dochodzić do n^2 .

95% pierwszych danych posortowanych			
Rozmiar	Merge[ms]	Quick[ms]	Intro[ms]
10000	2,1525	3,0120	2,1582
50000	11,4562	128,2370	13,5553
100000	23,0226	Limit!	28,6212
500000	123,3580	Limit!	195,2340
1000000	248,5450	Limit!	415,3250

Dla 95% pierwszych posortowanych, a danych pozostałych losowych, widzimy, że najszybszy jest MergeSort. Widzimy tendencję, że im większe będą ilości danych tym MergeSort będzie bardziej zyskiwał przewagę. Złożoność MergeSort i IntroSort to $O(n)$, a QuickSort może dochodzić do n^2 .

99% pierwszych danych posortowanych			
Rozmiar	Merge[ms]	Quick[ms]	Intro[ms]
10000	2,0944	9,0122	2,4838
50000	11,2501	55,4888	14,0467
100000	22,6836	Limit!	28,2467
500000	119,2990	Limit!	165,4030
1000000	244,4500	Limit!	347,3140



Dla 99% pierwszych posortowanych, a danych pozostałych losowych, widzimy, że najszybszy jest MergeSort. Widzimy tendencję, że im większe będą ilości danych tym MergeSort będzie bardziej zyskiwał przewagę. Złożoność MergeSort i IntroSort to $O(n)$, a QuickSort może dochodzić do n^2 .

Wnioski:

QuickSort jest algorytmem szybkim, lecz jest podatnym na awarie, jeśli dobierzemy niefortunnie dane może się okazać, że w programie wyskoczy błąd.

IntroSort jest poprawionym QuickSortem, przez zamienienie algorytmu na HeapSort jest on bardzo wydajny i odporny na awarie.

Merge okazuje się najwydajniejszym algorytmem dla ogromnych zbiorów danych. Przy Milionie elementów niejednokrotnie w testach był o 10% szybszy od IntroSorta.

Źródła:

Cormen T., Leiserson C.E., Rivest R.L., Stein C., Wprowadzenie do algorytmów, WNT
Drozdek A., C++. Algorytmy i struktury danych, Helion